Are NBFNet's Path Explanations Actually Faithful? A Deletion, Edit and Neuromodulation Benchmark on a Frozen Knowledge Graph Reasoner

Shadi Khodagholi
University of Central Florida
zahra.khodagholi@ucf.edu

Code and data: https://github.com/shadi97kh/nbfnet-path-faithfulness

Abstract

NBFNet explains each link prediction with a few paths through the knowledge graph. It finds them from the gradient of its score with respect to each edge and presents them as the reason for the answer. I test whether that is true. Instead of reading the explanation, I change the graph and run a frozen public checkpoint again: four graphs, two of them unseen in training, 800 queries the model ranks in its top ten, 15,447 single-path deletions, joint deletions, random-edge controls, edits that move the last hop of a path, and a test that turns whole relation types down like a dose. First, I found a bug. The released explanation code takes the gradient of a version of the model whose messages run backwards, so its gradient is off by about as much as the effect it predicts (median error 100%) and has the wrong sign on 21 of 50 random directions. A small fix brings the error down to 0.7%. Second, with or without the fix, the model does depend on the edges the explanation picks: deleting them removes between 0.259 and 0.858 of the right answer's probability, against at most 0.007 for as many random edges. Third, the order of the paths means little. Kendall's τ between that order and the measured effect is between -0.009 and 0.076 as released. The fix adds about 0.1, and integrated gradients on the same paths reach 0.270 to 0.489, still far from 1. Fourth, relation types often act like switches: only 0.030 to 0.290 of a relation's effect appears at half strength, where a straight line would give 0.5. The ULTRA paper's own checkpoint reproduces its reported accuracy and shows the same results. Whether the listed paths are separate evidence or one edge repeated depends on the graph, and the output never says which.

1. Introduction

NBFNet [41] predicts a missing link from the paths between two entities, and it explains each prediction with the same computation. It gives every edge a weight, runs the model, and takes the gradient of the score with respect to those weights. A beam search over the gradients then returns the top paths, and the paper presents them as the paths behind the answer. The explanation comes from the model itself, not from a second model fitted afterwards. That is what makes it appealing, and also why it is worth checking.

A gradient tells you what happens if an edge changes a little. Whether the model relied on that edge is a different question: what happens if the edge is gone. The two answers can disagree. I saw this in my earlier work on siRNA efficacy models, where saliency maps pointed at sequence positions that mattered much less than the maps suggested, and only perturbing the input showed it [19]. Paths are the main advantage of models like NBFNet over embeddings such as TransE [4]: an embedding cannot tell you why it made a prediction, and a path can. But that only helps if the path shown is the one the model used. A reader who checks a wrong path and finds it reasonable will trust the prediction for the wrong reason.

How this connects to the course. Topic 8 asked the same question for language models. Hase and colleagues found that where causal tracing [23] says a fact is stored barely predicts where an edit works, with a correlation near -0.13 [14]. MQuAKE found an editor that recalled 96.2% of the facts it wrote, while multi-hop accuracy on those facts fell from 40.5% to 7.0% [40], and edits that look reliable fall apart once the model writes freely [37]. Each time, a method passed an inspection test and failed an intervention test. I run the same kind of test on a graph reasoner. Here the intervention is a real knowledge edit, and unlike an edit to the weights, there is no doubt about whether it worked. The model is a Topic 2 method, NBFNet inside the ULTRA foundation model. The research track asks for a course method to be improved and compared one variable at a time. My variables are the direction of message passing inside the explanation, the attribution rule (the gradient or integrated

gradients [32]), the checkpoint, and whether the graph was seen in training. The ablation is the number of integration steps, random deletion is the control, and the reverse-fact queries in Section 7.7 are the error analysis. I also score the rankings with nDCG [17], the retrieval measure from the course.

Contributions. (1) A deletion and edit test of NBFNet's released explanation code on a frozen public checkpoint, with no training, over four graphs, with the number of queries chosen by a reliability check. (2) A bug in that code: the explanation takes the gradient of the model with every message reversed. I confirm it three ways and fix it. NBFNet's own PyTorch Geometric release has the same code pattern, which I read but did not run. (3) Evidence that the model depends on the selected edges on every graph, while the order of the paths says little, before and after the fix and on the ULTRA paper's own checkpoint. (4) Two measured improvements to that order, the direction fix and integrated gradients [32]. Neither makes it faithful. (5) A dose-response test on relation types, based on Bull *et al.*'s account of neuromodulation [5], which shows that they act like switches a local slope cannot see. (6) Evidence that whether the listed paths are many explanations or one depends on the graph, and that the output does not show which.

2. Background

NBFNet [41] turns the Bellman-Ford algorithm [3] into a neural network. To score a query $(h, r, ?)$, it starts the head entity with a vector for the relation and runs L rounds of relational message passing [29]. After that, each node's state sums up the paths of length at most L that reach it from h. I use ULTRA [13], whose entity-level reasoner is NBFNet unchanged (the class `EntityNBFNet` in its code). ULTRA adds a second NBFNet over a graph of relations. That pass turns the query relation into one vector per relation type, and in the entity graph every message along an edge of type c is multiplied by the vector for c, the DistMult operator [35]. ULTRA's checkpoints are public, so the model I explain is a published one.

I did not reimplement the explanation. `BaseNBFNet.visualize` runs the model with a weight on every edge, takes $\partial s / \partial w_e$ for every edge at every layer, beam-searches the highest-scoring paths under those gradients, and returns the top k. I call that function and test what it returns. Section 5 shows that it takes the gradient of a different model from the one it explains.

3. Related work

Faithfulness. Jacovi and Goldberg argue that an explanation has to be judged by what the model actually does, not by how convincing it looks [16]. ERASER [10] defined two common measures: comprehensiveness, the drop when the highlighted part is removed, and sufficiency, what is left when only that part is kept. I use both, on probabilities so that queries can be compared. Gradient saliency [31] can look right and still fail simple sanity checks [1], and removing the top-ranked inputs and retraining is a standard test for image models [15]. **Graph explanations.** GNNExplainer [38] learns a mask that keeps the prediction, and PGExplainer [20], SubgraphX [39] and GraphMask [30] followed. Most of this work produces explanations rather than testing them. The benchmarks that do test them mostly use synthetic graphs with a planted motif [2], which cannot tell whether a method matches what a model depends on in a real graph. **Knowledge graphs.** For embedding models, CRIAGE [26] and Kelpie [27] explain a prediction by the training facts whose removal changes it. That is the intervention I use, but on models with no paths to read. Rule learners [36], walk-based agents [8] and subgraph reasoners [33] also return paths or rules. NBFNet's paths come from its own computation, and A*Net [42] scales that design further. **Instability.** Models with the same MRR give different top-K predictions across random seeds [24], so an explanation from one run needs error bars. **Neuromodulation.** In the brain, a few broadcast signals change what a fixed circuit does without rewiring it [22]. Bull and colleagues model this as a small state that multiplies a network's connection strengths [5]. Such dose responses do not have to be monotone [7]. The same worry, that a flat local slope can hide a large overall effect, is the reason for integrated gradients [32].

4. Method

I write $s_G(h, r, t)$ for the model's score on graph G and $p_G(t \mid h, r)$ for the softmax of that score over all entities. I compare probabilities, not scores, because a score has no fixed scale across queries, and I report the right answer's rank next to it.

Conditions. *A, gradient*: call `visualize` and take its top paths in its own order. *B, single-path deletion*: delete each of the top 20 paths one at a time and measure the drop. This gives the order the gradient should have produced. *C, set level*: delete the top 5 paths together (comprehensiveness), and separately keep only their edges (sufficiency). *D, rewire*: point the last hop of the top path, the edge that reaches the predicted answer, at the second-ranked entity instead, together with its stored inverse, and measure how much the old answer loses and the new one gains.

E, relation channels. ULTRA has a structure like the one Bull *et al.* describe [5]. Its relation-level pass runs once per query, and its output multiplies every message of a relation type, so each relation type acts as a channel that reaches all of its edges at once. I give each channel c a gain g_c that scales every message along relation c and its inverse; $g = 1$ is the unchanged model. The gradient $\partial p / \partial g_c$ at $g = 1$, one backward pass for all channels, is the local slope. It

is NBFNet's edge-level signal added up over the channel, taken through the corrected code of Section 5. The dose response is p as g_c is turned down to 0.75, 0.5, 0.25 and 0. I measure it for every channel the top 5 paths use, for the 3 other channels the gradient ranks highest, and, at 0.5 and 0, for 3 random channels as a control. Then I turn all the explained channels down together. A gain of zero on a set of edges is the same as deleting them, so for each query I first set the gain to zero on exactly the shortlist's edges and check that this gives the deletion recorded in condition C. The largest gap over 550 queries is 0. Turning a channel down is a knowledge edit on a whole relation instead of a single fact.

Interventions are real edits. A deletion removes edge columns from `edge_index`, so the model runs on a graph that really lacks those facts. I keep ULTRA's relation graph fixed, so that removing a few edges cannot also change the relation vectors and add a second change to the measured drop.

Controls. *Random edges*: delete as many randomly chosen edges as the shortlist contains. *The bottom of the list*: the method's own lowest-ranked candidates give a baseline for its ranking within each query. *Answer degree*: if the answer has only one edge, any deletion looks decisive, so I record its degree and repeat the main result on answers with four or more edges. *Reverse facts*: some graphs keep symmetric relations, so the reverse of a query can already be in the training graph, which turns it into a one-hop lookup. I record this for every query. I added this control after finding the problem in a worked example (Section 7.7).

5. The explanation differentiates a different model

ULTRA, like NBFNet's PyTorch Geometric release, runs its layers with a fast sparse kernel, `rspmm`, whenever it can. When the edge weights need a gradient, which is exactly when `visualize` runs, each layer switches to the generic message passing of PyTorch Geometric [12]. The two do not agree on direction. The kernel sends the message on column e from `edge_index[1,e]` to `edge_index[0,e]`, and the generic code sends it from `edge_index[0,e]` to `edge_index[1,e]`. Training and prediction use the kernel, so that is the deployed model. The explanation takes the gradient of the other one: the same weights with every message running backwards.

I found this because integrated gradients [32] did not add up. Its attributions should sum to the change in p between the empty graph and the full one. With the released gradient they missed by a median of 63% of that change on the first 30 CoDEx-S queries, and by 15% with the corrected one. Three checks on 50 benchmark queries from CoDEx-S and NELL23k confirm it (Figure 1). First, the function the explanation differentiates gives scores up to 10.5 logits away from the deployed model. Run on the graph with every edge reversed, the same code matches the deployed scores to within 6×10^{-6}. Second, along a random direction in edge-weight space, the corrected gradient predicts the deployed model's change in p with a median error of 0.7%. The released one has a median error of 100% and the wrong sign on 21 of the 50 directions. The corrected error is not zero because a finite difference in single precision is itself approximate; with a step of 10^{-3} instead of 10^{-4} it is 0.2%. Third, for each query I take the explained relation channel where the two gradients disagree most and measure the slope of p against that channel's gain at 1. The released gradient has the wrong sign on 16 of these 50 channels, and the corrected one on 3.

The fix is to run `visualize` on the reversed graph and map each hop back to its original edge. I keep both versions. The released one is what anyone calling the function gets, so I report it as shipped, and I repeat the path benchmark with the corrected one (Figure 4). The channel gradients and integrated gradients use the corrected version only.

6. The benchmark

Model and graphs. I use ULTRA's `ultra_4g` checkpoint, 168,705 parameters, frozen, on two CPU cores. It was pretrained on FB15k-237, WN18RR, CoDEx-M and NELL-995. So WN18RR [9], built from WordNet [25], and FB15k-237 [34] were in its training data. CoDEx-S [28] and NELL23k [21], from NELL [6], were not, although each comes from the same source as a graph that was. Table 1 lists the graphs and the reproduction of link prediction. The reproduction comes first: if the checkpoint did not reproduce, the model I explain would not be the published one.

Queries. The test triples are not in the graph the model passes messages over, so an explanation cannot point at the answer itself. I take them in a fixed random order and keep a query when the model ranks the right answer in its top ten, since asking what a prediction relies on only makes sense when the prediction is right. For each query the beam search returns up to 20 candidate paths. I delete each of them separately, and the top 5 form the shortlist.

How many queries are enough? The model is deterministic: re-running recorded queries from a fresh clone of the code gives back every number, in every stage, to within 10^{-6}. So the only variation comes from which queries are drawn, and every interval below is a 95% bootstrap [11] over queries. To choose the sample size, I computed each main estimate on the first n queries for growing n. This is a fair sample because the order was fixed in advance (Figure 2). The estimates settle, and the bands narrow roughly as $1/\sqrt{n}$. The same analysis shows what a small study would have risked. On WN18RR, separate blocks of five queries give a "top path beats chance" score anywhere from -0.51 to 0.33, and the running estimate still had the opposite sign at 50 queries. At full size it is -0.028, within ± 0.052. When

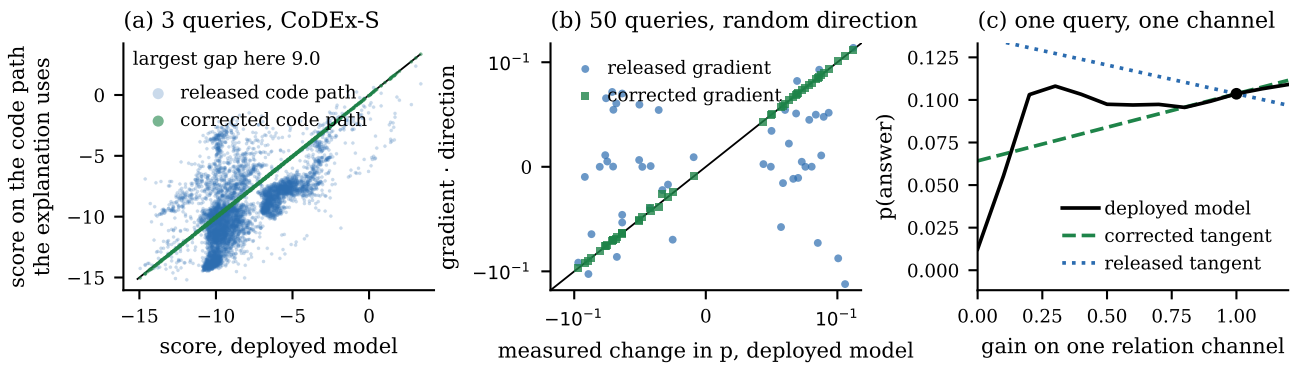


Figure 1. The released explanation code takes the gradient of a model with its messages reversed. (a) Every entity's score from the code the explanation differentiates, against the deployed model's score: as released (blue) and on the reversed graph (green). (b) Along one random direction in edge-weight space per query, the deployed model's measured change in p, by a central difference, against each gradient's prediction; the line is exact agreement. (c) One query and one relation channel: p as the channel's gain runs from 0 to 1.2, with each gradient's tangent at gain 1.

Table 1. The four graphs. MRR and Hits@k are filtered, both directions scored, with ULTRA's own evaluation code, on the number of test triples shown (filtered ranking [4]; all test triples except on FB15k-237). "Paper" is the MRR Galkin et al. report for ULTRA pre-trained on three graphs [13]. The `ultra_4g` checkpoint I use adds NELL-995 to that training set, so I expect close agreement rather than identical numbers; the "3g" column is that paper's own checkpoint, run here on the full test set. Explained queries are the test triples the frozen model ranks in its top ten, taken in a fixed random order set before any measurement.

graph	entities	relations	edges	`ultra_4g` saw it	test scored	MRR	H@10	paper	3g	queries	scanned
WN18RR	40,943	11	173,670	yes	3134	0.484	0.617	0.480	n/a	200	288
FB15k-237	14,541	237	544,230	yes	2000	0.371	0.558	0.368	n/a	200	400
CoDEx-S	2,034	42	65,776	held out	1828	0.464	0.665	0.472	0.472	200	274
NELL23k	22,925	200	50,890	held out	4952	0.258	0.431	0.239	0.241	200	396

the WN18RR queries are split at random into two halves, the two estimates of the shortlist deletion effect differ by at most 0.086 in 95% of splits.

7. Results

7.1. The model depends on the selected edges

This first result supports the method, and it holds on every graph. Deleting the shortlist removes 0.858 of the right answer's probability on WN18RR, 0.259 on FB15k-237, 0.427 on CoDEx-S and 0.368 on NELL23k (Table 2). Deleting the same number of random edges removes nothing measurable on any graph. So the gradient does point at a part of the graph the model depends on, which did not have to be true.

Keeping only the shortlist does not give a clean answer the other way. With only a few edges left, most other candidate answers are cut off too, so the right answer can gain probability just because it has fewer competitors. This happens on 8.0% of WN18RR queries, 36.5% of FB15k-237, 81.0% of CoDEx-S and 12.0% of NELL23k. I report the share that is kept, but I draw no conclusion about sufficiency from it. For a model that takes a softmax over every entity, deletion is a sound test of necessity, and keeping only a few edges is not a sound test of sufficiency.

7.2. As released, the order means almost nothing

If the ranking were faithful, Kendall's τ [18] between the method's order and the measured drop would be close to 1. It is -0.009 on WN18RR, 0.048 on FB15k-237, 0.076 on CoDEx-S and 0.041 on NELL23k. Figure 3 shows this directly: going down the method's own list, the effect of deleting each path stays flat. The simplest version of the question is how often the path ranked first is really the most damaging one to delete. On WN18RR the rate is 0.253, against a chance rate of 0.280, so no better than a random pick. On the other graphs it is at or a little above chance. This matches Hase *et al.*'s result in a different setting [14]: a number read from inside a model can correlate with what the model does and still not measure what it depends on.

7.3. Fixing the direction helps the order a little

Repeating the benchmark with the corrected explanation leaves the set-level result about the same (Figure 4). Its shortlist removes 0.461 of the right answer's probability on CoDEx-S and 0.394 on NELL23k, compared with 0.427 and 0.368 as released. The order does improve. On the queries

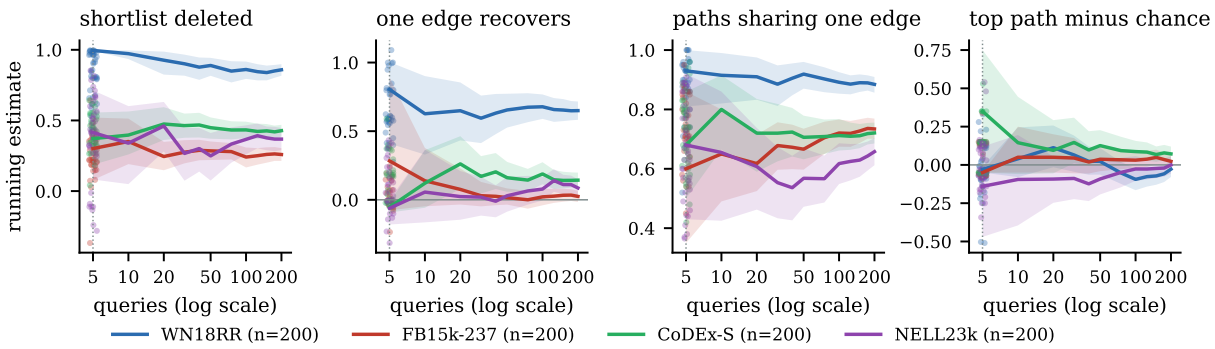


Figure 2. Each main estimate against the number of queries, as one unbroken line with its 95% bootstrap band, from $n = 5$ to the full sample. The dots at $n = 5$ are every separate block of five queries, which is what a five-query benchmark could have reported.

Table 2. Main results. Shares are of each query's own starting probability. Every row uses all 200 queries per graph except the corrected row on FB15k-237 (100), integrated gradients (queries where τ is defined, 200 and 160) and the channel rows (100, 50, 200 and 200). Brackets are 95% bootstrap intervals over queries. "Chance" is how often a random path from the list would be the most damaging one, counting ties as hits.

	WN18RR	FB15k-237	CoDEx-S	NELL23k
set level: does the model depend on the selected edges?				
delete shortlist, share of p removed	0.858 [0.813, 0.898]	0.259 [0.212, 0.303]	0.427 [0.386, 0.470]	0.368 [0.290, 0.444]
delete as many random edges	0.000	0.000	0.000	0.007
ranking: does the order mean anything?				
Kendall τ, rank against effect	−0.009 [−0.054, 0.037]	0.048 [0.010, 0.085]	0.076 [0.043, 0.108]	0.041 [−0.023, 0.102]
corrected explanation	0.082 [0.036, 0.128]	0.153 [0.098, 0.208]	0.185 [0.146, 0.224]	0.103 [0.042, 0.160]
integrated gradients [32]	0.270 [0.188, 0.353]	0.432 [0.348, 0.513]	0.489 [0.445, 0.531]	0.445 [0.372, 0.516]
top path is the most damaging	0.253	0.080	0.125	0.306
chance	0.280	0.057	0.053	0.307
relation channels, corrected gradient				
silence strongest explained channel	0.995	0.770	0.895	0.895
silence a random channel	0.020	0.000	0.002	0.101
share of effect present at half gain	0.29	0.23	0.19	0.03
gradient has the sign of silencing	59.2%	64.9%	60.8%	73.9%
strongest channel is the query relation	92.0%	85.7%	96.5%	71.5%
redundancy: many explanations or one?				
paths sharing the busiest edge	0.884	0.734	0.721	0.658
distinct edges / edges listed	0.248	0.484	0.512	0.295
that one edge recovers	0.650 [0.581, 0.716]	0.025 [−0.014, 0.066]	0.143 [0.093, 0.197]	0.086 [0.026, 0.147]
structure of the queries				
reverse fact already in graph	57.5%	0.0%	13.0%	4.5%
median answer degree	13	854	320	92

where τ is defined for both versions (200 on CoDEx-S, 162 on NELL23k; Table 2 averages over all of them), Kendall's τ rises from 0.076 to 0.185 on CoDEx-S, a gain of 0.109 [0.064, 0.155], and from 0.030 to 0.111 on NELL23k, a gain of 0.081 [0.010, 0.151]. On the seen graphs the gain is 0.082 and 0.076. So the bug explains part of why the order was poor, but not most of it. The correct gradient of the correct model still ranks paths far from their measured effect, while ranking by the measured drops themselves would score 1. ULTRA's three-graph checkpoint, the model in the ULTRA paper, gives the same picture, with τ of 0.182 and 0.109. It also reproduces that paper's MRR: 0.472 against 0.472 on CoDEx-S, and 0.241 against 0.239 on NELL23k (Table 1).

7.4. Integrated gradients rank better, but not well

The gradient is a slope at the point where every edge weight is one. Integrated gradients average that slope along the way from the empty graph to the full one [32]. I re-rank

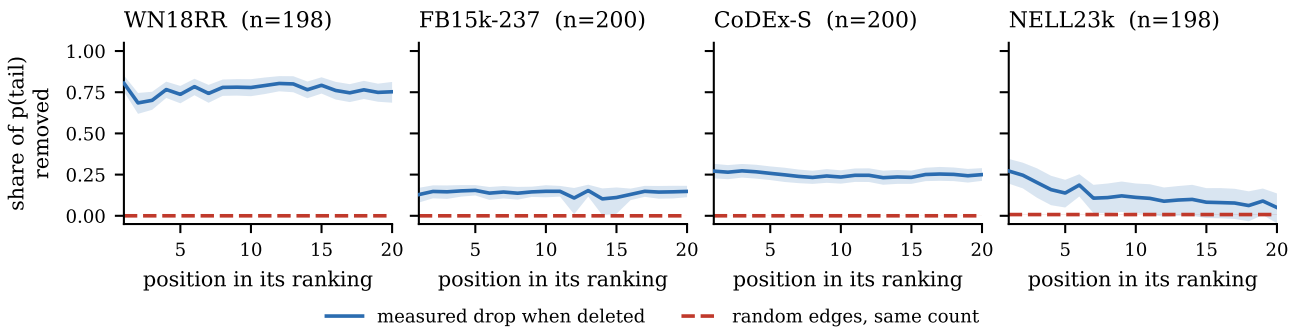


Figure 3. What the method ranks first is not what matters most. The horizontal axis goes down the method's own order of its 20 candidates; the vertical axis is the share of the right answer's probability that deleting that path removes, with a 95% bootstrap band. A faithful ranking would fall from left to right. The dashed line deletes the same number of random edges.

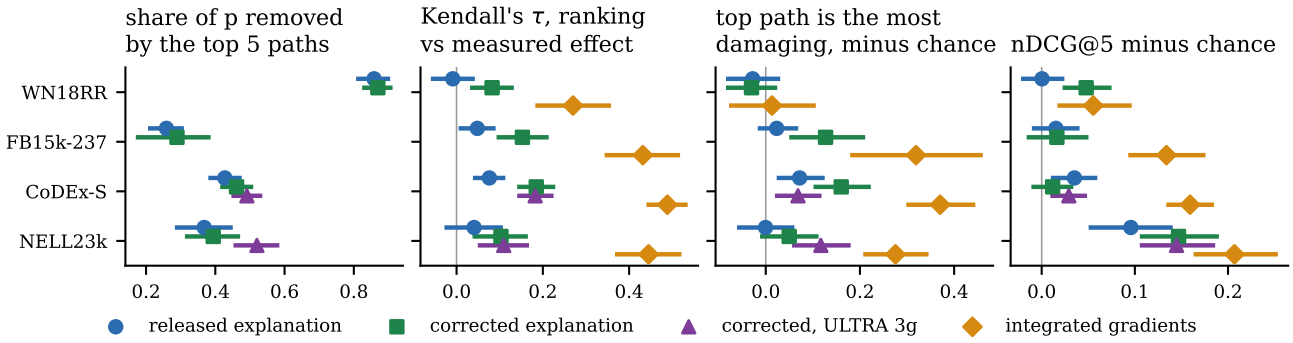


Figure 4. The main numbers, changing one variable at a time: the released explanation, the direction-corrected one, the corrected one on ULTRA's three-graph checkpoint [13], and integrated gradients [32] re-ranking the corrected explanation's own candidates. Points are means over queries, lines 95% bootstrap intervals. Integrated gradients re-ranks a fixed set of candidates, so it has no joint-deletion value. nDCG@5 [17] scores the first five paths with each path's measured drop as its relevance, minus the score of a random order.

the corrected explanation's own candidates by the integrated gradients summed over each path's edges. This raises τ on every graph (Figure 4), by 0.179 to 0.305 on the same queries, to between 0.270 and 0.489. On CoDEx-S it goes from 0.185 to 0.489, a paired gain of 0.304 [0.257, 0.350], and how often the top path is the most damaging one, above chance, goes from 0.160 to 0.370. Summed over a relation channel's edges, integrated gradients get the sign of that channel's silencing effect right on 90.4% of the channels that matter on CoDEx-S and 86.3% on WN18RR, compared with 60.8% and 59.7% for the gradient, both averaged per query as in Table 2.

The integral is only approximated, so I ablated its one setting, the number of steps. With 16 steps, the attributions add up to the total change only to within a median of 15% on the first 50 CoDEx-S queries. With 32 and 64 steps the error falls to 11% and 5%, while τ stays at 0.467, 0.472 and 0.456, and the 16-step and 64-step orders agree with $\tau = 0.903$. So the gain does not come from a coarse integral. It costs 16 backward passes instead of one, and it is the largest improvement in the study. Still, τ stays at or below 0.5: the order of the paths is only loosely tied to what deleting them does.

7.5. Relation channels are switches, not dials

Turning a channel down shows why a slope can mislead (Figure 5). Silencing the strongest channel the explanation uses removes 0.895 of the right answer's probability on CoDEx-S and 0.895 on NELL23k, while a random channel removes 0.002 and 0.101 (Table 2). But the response is not gradual. For channels whose silencing matters, only 0.19 of the full effect is there at half gain on CoDEx-S, where a straight line would give 0.5. 51.7% of them have less than a quarter of it, and 35.6% change direction somewhere along the way, a non-monotone response like the one reported for dopamine [7]. Turning every explained channel down to half at once removes nothing measurable on CoDEx-S (-0.027 [-0.071, 0.017]) and raises p on NELL23k (-0.181), compared with 0.909 at zero on CoDEx-S. WN18RR is the exception: there, half gain on the explained channels together already

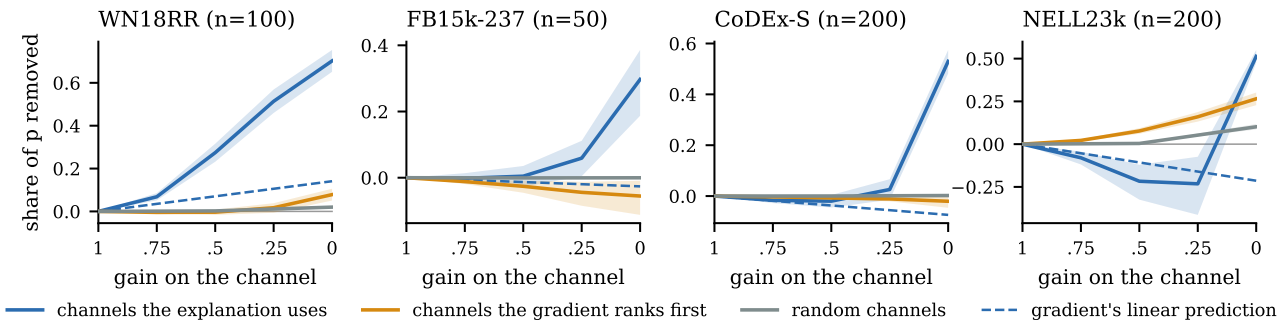


Figure 5. Turning one relation channel down from gain 1 to 0. Curves are the share of the right answer's probability removed, averaged over queries, with 95% bootstrap bands, for the channels the explanation's paths use, the other channels the corrected gradient ranks highest, and random channels. The dashed line is the gradient's straight-line prediction for the explained channels.

removes 0.489, about what a straight line gives. A slope taken at gain 1 cannot see a switch near 0. The corrected gradient has the sign of the silencing effect for only 60.8% of these channels on CoDEx-S and 73.9% on NELL23k, where chance is about half. The channel that matters most is usually the query's own relation, from 71.5% of queries on NELL23k to 96.5% on CoDEx-S. The paths do name that relation, as one of their hops. What they cannot show is that it works like a switch. This is the Topic 8 result again, one level up: the gradient finds a channel, and finding it does not predict what editing it does.

7.6. One edge or many depends on the graph

The shortlist is shown as several separate paths. On WN18RR it is close to one. A single edge appears in 0.884 of the listed paths, only 0.248 of the listed edges are distinct, and deleting that one edge gives 0.650 of the effect of deleting the whole shortlist. One could object that these answers are poorly connected, so any deletion cuts them off. That is not the case: the median answer degree is 13, and the result holds for answers with four or more edges (0.613).

The other graphs look different. On FB15k-237 the busiest edge appears in 0.734 of paths, and deleting it gives 0.025 of the effect, with an interval that includes zero. There the paths really are different routes. The difference comes from the graphs themselves (Tables 1 and 2). WN18RR is sparse, has 11 relation types and keeps symmetric relations, so 57.5% of its queries have their reverse fact in the graph. FB15k-237 has 237 relation types, a median answer degree of 854, and 0.0% of queries with a reverse fact. The held-out graphs fall between the two, which lets me check the pattern on graphs the model never saw.

7.7. A control I found by reading one example

While I was building a worked example on WN18RR, the top "path" came back as a single hop, and that hop was the stored reverse of a training fact. Table 4 shows the first such query in the benchmark's order. The model is asked what *sociology* is derivationally related to and answers *sociologist*, because the training graph already has that fact the other way round. The explanation offers five paths. Two of them go from sociology to *mores* and straight back before taking the same reverse hop, and deleting that one hop alone does everything that deleting all five paths does. This is not leakage, since the test edge itself is never in the graph, but it changes what there is to explain, so Table 3 splits WN18RR on it. Two findings hold on both halves: the paths go through one edge, and the ranking is no better than chance. What changes is how much that edge carries, 0.807 with the reverse fact present and 0.395 without, and almost all of the rewire effect. The first column is the most important result here. On those queries the model is doing a one-hop lookup, and the explanation returns up to 20 paths of up to six hops. Nothing in that output says the answer came from one reverse lookup, which is exactly what a reader would want to know.

Table 3. WN18RR split on whether the query's reverse fact is in the training graph. The forward edge is never present (0.0%).

	reverse present (115 queries)	reverse absent (85 queries)
paths sharing one edge	0.894	0.872
Kendall τ	−0.012	−0.003
top path is best	0.157	0.392
chance	0.217	0.372
one edge recovers	0.807	0.395
rewire, old loses	0.2808	0.0017

7.8. Rerouting a path removes the answer instead of moving it

If the top path were how the model reached its answer, pointing its last hop at another entity should move the answer there. On WN18RR the old answer loses 0.1622 while the new target gains only 0.0282, almost all of it on reverse-fact queries. On the other graphs neither moves much. The edit takes support away without moving it, the same portability

Table 4. A worked example, the first query in the benchmark's fixed order that meets the rule in its heading, re-run under the benchmark protocol. Arrows show the direction of the stored fact, so a leftward arrow is a hop taken backwards along a reverse edge. Names are from the WN18RR text release.

WN18RR, first query with its reverse fact in the graph. Query: sociology − derivationally related form → ? answer **sociologist**, rank 1, $p = 0.196$	
path 1 (1 hop)	sociology ← derivationally related form − sociologist
path 2 (3 hops)	sociology ← synset domain topic of − mores
	mores − synset domain topic of → sociology
	sociology ← derivationally related form − sociologist
paths 3 to 4	longer loops that end in the same last hop as path 1
delete top 5 (6 edges)	p 0.196 → 0.000, rank 1 → 24859
delete busiest edge only	p 0.196 → 0.000, rank 1 → 24874: 100% of that effect
	that edge, in 4 of 5 paths: sociologist − derivationally related form → sociology

failure MQuAKE found [40], but here with an exact edit.

8. Discussion

The bug is the most concrete finding, and the lesson is general. A gradient explanation describes whatever code was differentiated, and here that was not the code that makes the predictions. Nothing in the output looks wrong, and the deletion tests did not catch it. What caught it was checking the gradient against the model's own response: first the completeness of integrated gradients [32], then the scores and a finite difference.

The gradient finds a region of the graph the model depends on, and the random-edge control shows this is not luck. As released, it says little or nothing about the order inside that region, and even after the fix it ranks poorly. Whether it splits the region into separate pieces of evidence depends on the graph. A heat map over edges and a ranked list of reasoning chains promise different things. NBFNet's gradient delivers the first but is presented as the second.

Five paths that agree look like five pieces of evidence. If they all go through the same edge, they are really one, and a reader who checks two of them has learned much less than they think. The output looks the same either way. Five changes would help: run the explanation on the reversed graph; rank with integrated gradients [32], at 16 backward passes, if an order is going to be shown; report the busiest edge and check it with one deletion; print the number of distinct edges next to any list of paths; and flag queries whose reverse fact is already in the graph.

None of this means the model is broken. Its accuracy is in line with the ULTRA paper, and it does need the edges its gradient selects. What fails is the explanation layer. First, the gradient goes through the wrong code. Then, once that is fixed, it is a real but rough signal over edges, shown as a ranked list of separate reasoning chains that it does not support. The channel experiment suggests why a better gradient is not enough: much of the answer depends on relation channels that act as switches, and a slope cannot see a switch.

9. Limitations

Overlapping paths. Deleting a path also affects other paths that share its edges, so a single-path drop is not only that path's. The redundancy rows of Table 2 measure this, and the set-level and single-edge numbers do not depend on it. **One model family** on four graphs: the claim is about this family's explanations, not about path reasoning in general. **Correct predictions only**: how the explanation behaves on the model's mistakes is a separate question. **Smaller runs on the seen graphs**: the corrected benchmark has 100 queries on FB15k-237, the channel experiment 100 on WN18RR and 50 on FB15k-237, and integrated gradients 86 and 50, against 200 elsewhere. The second checkpoint was run on the held-out graphs only. **NBFNet-PyG**: its layer code has the same fallback, but I ran the checks on ULTRA only. **WN18RR node merges**: the version PyTorch Geometric loads merges 161 unrelated synsets (0.4%) into shared nodes. The checkpoint was trained on that graph, so I left it as is.

10. Conclusion

I tested whether the paths NBFNet reports are the paths it relies on, by editing the graph of a frozen checkpoint instead of reading its gradients, on 800 queries over four graphs. The released explanation takes the gradient of the model with its messages reversed; once fixed, it matches the deployed model. Either way, the model depends on the selected edges, but their order says little. The fix and integrated gradients each improve the order, and neither makes it faithful. Pointing at the right part of the graph is not the same as explaining the answer, and only changing the graph shows the difference.

References

[1] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. 2

[2] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju, and

Marinka Zitnik. Evaluating explainability for graph neural networks. *Scientific Data*, 10, 2023. 2

[3] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958. 2

[4] Antoine Bordes, Nicolas Usunier, Alberto García-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2787–2795, 2013. 1, 4

[5] Matthew S. Bull, Po-Chen Kuo, Andrew L. Smith, and Michael A. Buice. Volume transmission implements context factorization to target online credit assignment and enable compositional generalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2

[6] Andrew Carlson, Justin Betteridge, Bryan Kisiel, Burr Settles, Estevam R. Hruschka Jr., and Tom M. Mitchell. Toward an architecture for never-ending language learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2010. 3

[7] Roshan Cools and Mark D'Esposito. Inverted-U-shaped dopamine actions on human working memory and cognitive control. *Biological Psychiatry*, 69(12):e113–e125, 2011. 2, 6

[8] Rajarshi Das, Shehzaad Dhuliawala, Manzil Zaheer, Luke Vilnis, Ishan Durugkar, Akshay Krishnamurthy, Alex Smola, and Andrew McCallum. Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2018. 2

[9] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2D knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. arXiv:1707.01476; source of the FB15k-237 and WN18RR splits. 3

[10] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4443–4458, 2020. arXiv:1911.03429. 2

[11] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. 3

[12] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. 3

[13] Mikhail Galkin, Xinyu Yuan, Hesham Mostafa, Jian Tang, and Zhaocheng Zhu. Towards foundation models for knowledge graph reasoning. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.04562. 2, 4, 6

[14] Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2301.04213. 1, 4

[15] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. A benchmark for interpretability methods in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 2

[16] Alon Jacovi and Yoav Goldberg. Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness? In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4198–4205, 2020. 2

[17] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. 2, 6

[18] Maurice G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1/2):81–93, 1938. 4

[19] Zahra Khodagholi and Niloofar Yousefi. Validating interpretability in siRNA efficacy prediction: A perturbation-based, dataset-aware protocol. In *MLGenX Workshop, International Conference on Learning Representations (ICLR)*, 2026. arXiv:2602.10152. 1

[20] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. Parameterized explainer for graph neural network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2

[21] Xin Lv, Xu Han, Lei Hou, Juanzi Li, Zhiyuan Liu, Wei Zhang, Yichi Zhang, Hao Kong, and Suhui Wu. Dynamic anticipation and completion for multi-hop reasoning over sparse knowledge graph. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5694–5703, 2020. 3

[22] Eve Marder. Neuromodulation of neuronal circuits: Back to the future. *Neuron*, 76(1):1–11, 2012. 2

[23] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1

[24] Guillaume Méroué, Fabien Gandon, and Pierre Monnin. Link prediction or perdition: the seeds of instability in knowledge graph embeddings. *arXiv preprint*, 2026. arXiv:2606.03365. 2

[25] George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41, 1995. 3

[26] Pouya Pezeshkpour, Yifan Tian, and Sameer Singh. Investigating robustness and interpretability of link prediction via adversarial modifications. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 3336–3347, 2019. 2

[27] Andrea Rossi, Donatella Firmani, Paolo Merialdo, and Tommaso Teofili. Explaining link prediction systems based on knowledge graph embeddings. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 2062–2075, 2022. 2

[28] Tara Safavi and Danai Koutra. CoDEx: A comprehensive knowledge graph completion benchmark. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 8328–8350, 2020. 3

[29] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web: European Semantic Web Conference (ESWC)*, pages 593–607, 2018. 2

[30] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. Interpreting graph neural networks for NLP with differentiable

edge masking. In *International Conference on Learning Representations (ICLR)*, 2021. 2

[31] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014. 2

[32] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, pages 3319–3328, 2017. 2, 3, 5, 6, 8

[33] Komal K. Teru, Etienne G. Denis, and William L. Hamilton. Inductive relation prediction by subgraph reasoning. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 9448–9457, 2020. 2

[34] Kristina Toutanova and Danqi Chen. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, pages 57–66, 2015. 3

[35] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations (ICLR)*, 2015. 2

[36] Fan Yang, Zhilin Yang, and William W. Cohen. Differentiable learning of logical rules for knowledge base reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[37] Wanli Yang, Fei Sun, Jiajun Tan, Xinyu Ma, Du Su, Dawei Yin, and Huawei Shen. The mirage of model editing: Revisiting evaluation in the WILD. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2502.11177. 1

[38] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. GNNExplainer: Generating explanations for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1903.03894. 2

[39] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. On explainability of graph neural networks via subgraph explorations. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 12241–12252, 2021. 2

[40] Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and Danqi Chen. MQuAKE: Assessing knowledge editing in language models via multi-hop questions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2305.14795. 1, 8

[41] Zhaocheng Zhu, Zuobai Zhang, Louis-Pascal Xhonneux, and Jian Tang. Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2106.06935. 1, 2

[42] Zhaocheng Zhu, Xinyu Yuan, Mikhail Galkin, Sophie Xhonneux, Ming Zhang, Maxime Gazeau, and Jian Tang. A*Net: A scalable path-based reasoning approach for knowledge graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2206.04798. 2